Klassifikation von Kleidungsstuecken mit einem CNN

Projektpraesentation basierend auf cnn_3.ipynb

Vades Ngounouo Djoko Lister Melaguie

Deep Learning

26. Juni 2026

Projektname und Ziel

Projektname: ClothingCNN - Bildklassifikation von Kleidungsstuecken

Ziel:

- Aufbau und Training eines Convolutional Neural Networks zur Klassifikation von 5 Kleidungsklassen.
- Robuste Generalisierung durch Datenaugmentation, Learning-Rate-Scheduling und Early Stopping.
- Analyse der Fehlerfaelle mittels Konfusionsmatrix und Fehlklassifikationsbeispielen.

Klassen: long-sleeved, pants, shorts, socks, t-shirt

Datensatz: Umfang und Struktur

- Gesamtanzahl Bilder: **3899**
- Bildbreite: min 224, max 5712, Mittelwert 1209.5, Std 978.5
- Bildhoehe: min 224, max 5712, Mittelwert 1236.3, Std 766.0
- Seitenverhaeltnis (B/H): min 0.405, max 2.248, Mittelwert 0.977
- Farbkanaele: **100% RGB** (3899 Bilder)

Datensplit:

- Train: 3119 (80%)
- Validation: 389 (10%)
- Test: 391 (10%)

Klassenverteilung

Klasse	Anzahl Bilder
long-sleeved	787
pants	783
shorts	767
socks	781
t-shirt	781
Gesamt	**3899**

- Ungleichgewichtsverhaeltnis (max/min): **1.026**
- Interpretation: Klassen sind gut ausbalanciert, keine Klassengewichtung in der Loss notwendig.

Vorverarbeitung und Augmentierung

Train-Transformationen:

- Resize(256), RandomResizedCrop(224, scale=[0.8,1.0])
- RandomHorizontalFlip(0.5)
- RandomRotation(10)
- ColorJitter(brightness/contrast/saturation = 0.2)
- Normalize(mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225])

Validation/Test:

- Resize(256), CenterCrop(224), Normalize(...)

Nutzen: Hoehere Robustheit und bessere Generalisierung bei variierenden Perspektiven/Beleuchtung.

Modellarchitektur (ClothingCNN)

- Eingabe: $3 \times 224 \times 224$
- Block 1: Conv2d(3,32,3,pad=1) $\rightarrow$ BatchNorm2d(32) $\rightarrow$ ReLU $\rightarrow$ MaxPool2d(2)
- Block 2: Conv2d(32,64,3,pad=1) $\rightarrow$ BatchNorm2d(64) $\rightarrow$ ReLU $\rightarrow$ MaxPool2d(2)
- Block 3: Conv2d(64,128,3,pad=1) $\rightarrow$ BatchNorm2d(128) $\rightarrow$ ReLU $\rightarrow$ MaxPool2d(2)
- Block 4: Conv2d(128,256,3,pad=1) $\rightarrow$ BatchNorm2d(256) $\rightarrow$ ReLU $\rightarrow$ MaxPool2d(2)
- Global Average Pooling: AdaptiveAvgPool2d(1,1)
- Klassifikator: Dropout(0.3) $\rightarrow$ Linear(256,256) $\rightarrow$ ReLU $\rightarrow$ Dropout(0.2) $\rightarrow$ Linear(256,5)
- Ausgabe: 5 Klassenlogits

Forward-Check: Eingabe [4, 3, 224, 224] $\rightarrow$ Ausgabe [4, 5]

Trainierbare Parameter pro Schicht

Schicht	Trainierbare Parameter
Conv2d(3,32,3x3)	896
BatchNorm2d(32)	64
Conv2d(32,64,3x3)	18496
BatchNorm2d(64)	128
Conv2d(64,128,3x3)	73856
BatchNorm2d(128)	256
Conv2d(128,256,3x3)	295168
BatchNorm2d(256)	512
Linear(256,256)	65792
Linear(256,5)	1285
Gesamt	**456453**

Hinweis: Dropout, ReLU, MaxPooling und GAP haben keine trainierbaren Parameter.

Trainingsparameter und Callbacks

Training-Setup:

- Device: CPU (kein GPU-Accelerator erkannt)
- Batch-Size: 32
- Epochen: 50 (Loop), W&B-Config auf 100 gesetzt
- Loss-Funktion: CrossEntropyLoss
- Optimierer: Adam ($lr = 3 \cdot 10^{-4}$, weight_decay=10^{-4})
- Scheduler: ReduceLROnPlateau (factor=0.5, patience=3, min_lr=10^{-6})

Callback: Early Stopping

- patience = 7, min_delta = 10^{-4}
- Speichert bestes Modell in best_model.pth

Trainingsergebnisse

- Bestes Validierungsmodell: **Epoche 46**
- Beste Val-Loss: **1.1326**
- Letzte Epoche (50):
  - Train-Loss: 1.0305
  - Train-Accuracy: 59.92%
  - Val-Loss: 1.1365
  - Val-Accuracy: 54.24%
- Lernrate wurde schrittweise reduziert bis ca. $1.87 \cdot 10^{-5}$

Testergebnisse und Konfusionsmatrix

- Gesamtgenauigkeit Testset: **53.71%**
- Klassenspezifische F1-Scores:

Klasse	**Precision**	**Recall**	**F1**
long-sleeved	0.510	0.298	0.376
pants	0.559	0.747	0.639
shorts	0.373	0.322	0.345
socks	0.654	0.726	0.688
t-shirt	0.515	0.554	0.534

Herausforderungen und Loesungsansaetze

Beobachtete Herausforderungen:

- Visuelle Aehnlichkeit zwischen Klassen (z.B. shorts vs. pants, t-shirt vs. long-sleeved)
- Unterschiedliche Bildausschnitte und Perspektiven
- Informationsverlust durch mehrfache Pooling-Schritte (feine Texturdetails)

Umgesetzte Loesungen:

- Datenaugmentation (Flip, Rotation, ColorJitter, RandomResizedCrop)
- Adaptive Lernrate (ReduceLROnPlateau)
- Early Stopping und Best-Checkpoint-Reload
- Detaillierte Fehleranalyse mit Konfusionsmatrix und Fehlklassifikationen

Fazit und naechste Schritte

Fazit:

- Das CNN lernt sinnvolle Merkmale und erreicht **53.71%** Testgenauigkeit auf 5 Klassen.
- Staerkste Klassen: socks und pants (hoher Recall/F1).
- Schwierigere Klassen: shorts und long-sleeved.

Moegliche Verbesserungen:

- Transfer Learning (z.B. ResNet, EfficientNet)
- Feinere Augmentierungsstrategie und hoehere Eingangsaufloesung
- Hyperparameter-Tuning und ggf. class-spezifische Loss-Strategien
- Sicheres Secret-Handling (W&B API-Key ueber Umgebungsvariablen statt im Notebook)